

From Keywords to Understanding

The Journey of Cataloguing 26,246 Datasets from HDX to RDLS

Regex --> Regex + LLM --> LLM-First

Benny Istanto
Risk Data Librarian
bistanto@worldbank.org

The Mission

Transform HDX metadata into the Risk Data Library Standard (RDLS)

RDLS classifies risk datasets into four components - HEVL:

H Hazard	Flood depth grids, seismic intensity maps, drought indices, storm surge models, etc.
E Exposure	Building inventories, population grids, facility registries, infrastructure datasets, etc.
V Vulnerability	Damage and fragility functions, vulnerability curves, loss ratios by asset type, etc.
L Loss	Casualties, displacement counts, economic damage, facility operational status, etc.

Reference: <https://docs.riskdatalibrary.org/en/latest/reference/schema/#>

26,246 HDX datasets - ~12,594 potential RDLS candidates - Manual review: \$2M+ and months of expert time

Activity 1

The Regex Pipeline

Deterministic. **Fast.** 12,594 records in under 60 seconds.

Act 1: How Pattern Matching Works

400+ regex patterns mapped to RDLs codelists - every metadata field scanned and scored

signal_dictionary.yaml (excerpt)

```
hazard:
- pattern: "flood.?(depth|inundation|extent|map)"
  type: flood | process: fluvial_flood | confidence: high
- pattern: "earthquake|seismic|ground.motion"
  type: earthquake | confidence: high
exposure:
- pattern: "building.?(stock|footprint|inventory|count)"
  category: buildings | confidence: high
- pattern: "population.?(grid|density|distribution)"
  category: population | confidence: high
loss:
- pattern: "casualt|mortalit|death.toll|killed"
  confidence: medium
- pattern: "displacement|displaced.person|IDP"
  confidence: medium
```

Scoring - 'Global Flood Hazard Map'

Title:	'flood'	+0.8	hazard/flood
	'hazard'	+0.6	hazard
Tags:	'floodplain'	+0.5	hazard/flood
Desc:	'inundation'	+0.7	hazard/flood
	'return per.'	+0.8	probabilistic

Hazard score:	3.7	>>	threshold 0.5	OK
Exposure:	0.0	<<	threshold	--
Loss:	0.0	<<	threshold	--

Output: Hazard only. Correct.

Works perfectly when metadata accurately describes data content. Processed 12,594 datasets in < 60 seconds at zero API cost.

What about datasets where the metadata describes the disaster context rather than the actual data content?

Act 1: The Content-vs-Context Ceiling

Pattern matching cannot distinguish what a dataset IS ABOUT from what it CONTAINS

The trigger mechanism - 'Kenya Drought-Related Key Figures' <https://data.humdata.org/dataset/c7e5d29e-f5e8-4d8b-a437-f784fcfd6103>

```
Title keywords matched:
"drought"    --> Hazard (drought, intensity: SPI:-) [FABRICATED]
"Kenya"      --> country = KEN

Description keywords matched:
"population" --> Exposure (population, count)      [FABRICATED]
"affected"   --> Vulnerability                      [FABRICATED]
"mortality"  --> Loss                               [CORRECT]

Pipeline output: H + E + V + L (all four components)

Reality: files contain ONLY displacement counts and mortality figures.
        Three components entirely invented by pattern matching.
```

Scale of the problem

Issue	Count	% of total
Records with phantom components	2,313	18.4%
Fabricated hazard blocks	~2,100	16.7%
Fabricated exposure blocks	~1,800	14.3%
Records mis-labelled as all 4 components	~400	3.2%

These 2,313 records contained fabricated hazard blocks with template descriptions and invented measurement parameters that did not exist in the underlying files.

Act 1: The Diagnostic Case - Kenya Drought

Dataset: 'Kenya Drought-Related Key Figures' (HDX)

Attribute	Regex output (wrong)	Reality (correct)
Classification	Hazard + Exposure + Vulnerability + Loss	Loss only
Hazard block	"drought hazard data for Kenya", intensity: SPI:-	No drought index in any file
Exposure block	Template from "population" keyword	No asset or population inventory
Vulnerability	Template from "affected" keyword	No fragility functions
Loss block	Displacement counts, mortality figures	Displacement counts, mortality figures

Generated (wrong)

```
"hazard": {
  "event_sets": [{
    "hazards": [{"type": "drought",
                  "intensity_measure": "SPI:-"}]
  }]
},
"exposure": [{"category": "population"}],
"vulnerability": {"functions": {...}}
```

Should have been

```
"risk_data_type": ["loss"],
"loss": {
  "losses": [{
    "hazard": {"type": "drought"},
    "asset_category": "population",
    "impact_metric": "displaced_count"
  }, {
    "impact_metric": "mortality_count"
  }]
}
```

Act 1: Results and Verdict

Metric	Value
Total candidates processed	12,594
Processing time	< 60 seconds
API cost	USD 0
Correctly classified	~10,281 (81.6%)
Over-classified (phantom components)	2,313 (18.4%)
Return periods captured	0 (no mechanism)
Multi-exposure records generated	0 (no mechanism)
Multi-loss entries generated	0 (no mechanism)

What worked well

- + Fast: 12,594 records in under 60 seconds
- + Deterministic and fully reproducible
- + Zero API cost
- + Correctly handled ~82% of all records

The fundamental limit

Pattern matching cannot cross the semantic boundary between what a dataset is about and what it actually contains. For HDX, this boundary affects 1 in 5 records. No additional patterns can fix it - the failure mode is architectural.

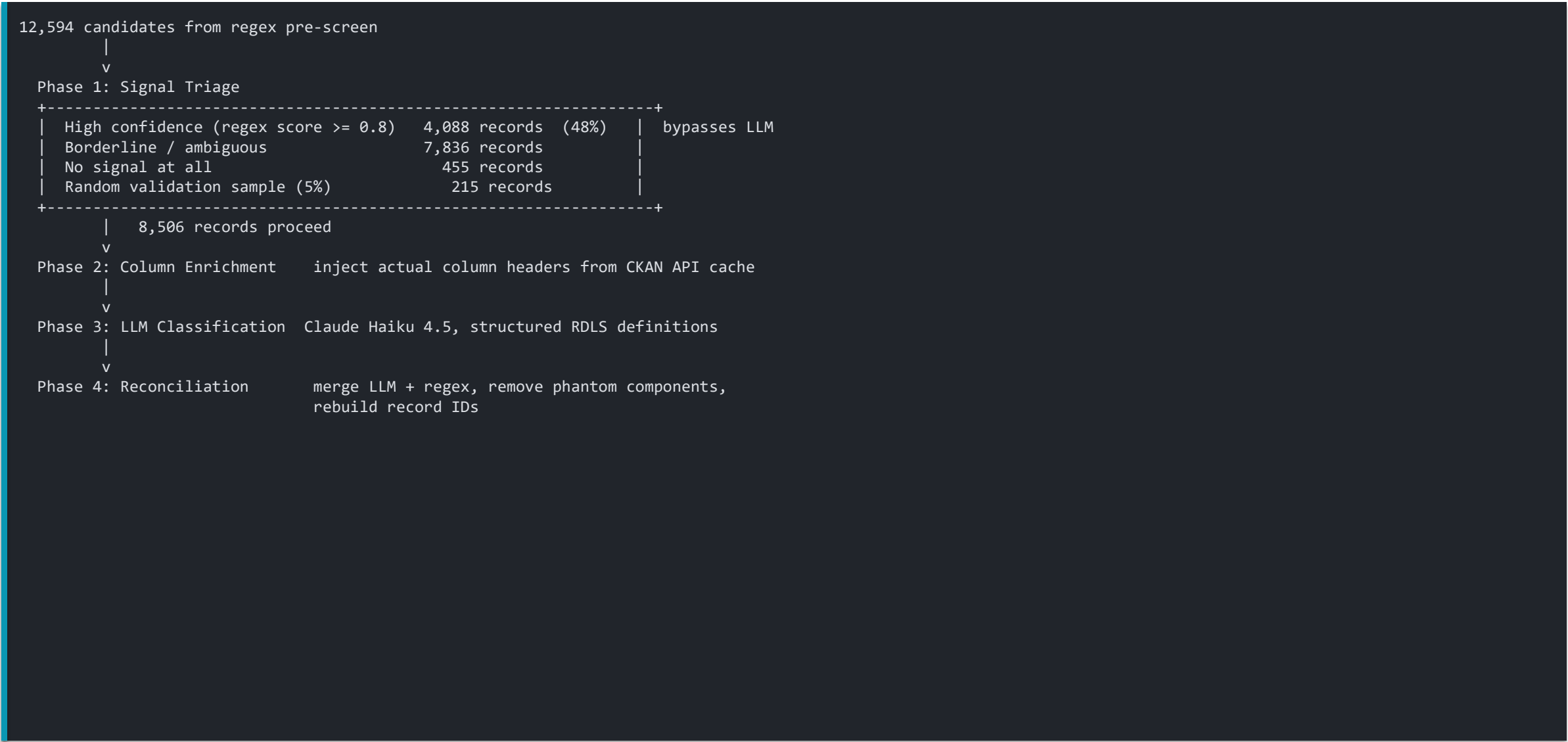
Activity 2

The Hybrid Pipeline

Regex for clear-cut cases. LLM where it matters. Column headers as evidence.

Act 2: The Four-Phase Hybrid Design

Regex handles 48% of records at zero cost; LLM reviews the ambiguous 52%



Act 2: The Column Enrichment Breakthrough

Actual data column headers - the single most impactful design decision

Without column enrichment (title + tags only)

```
Dataset: Somalia Flood Impact Assessment

Title: Somalia Flood Impact Assessment
Tags:  flood, Somalia, displacement,
      humanitarian, affected population
Desc:  'Assessment of flood impacts...'

LLM sees: flood + population + affected
         --> H + E + L ? (ambiguous)
```

With column enrichment (+ actual data columns)

```
Dataset: Somalia Flood Impact Assessment
...same metadata...

Column headers from resource files:
affected_population.csv:
  Region, District, Flood_Date,
  People_Affected, People_Displaced,
  Households_Affected, Deaths,
  Injuries, Houses_Damaged_Total,
  Houses_Damaged_Severe

LLM sees: displacement + deaths + damage
         --> Loss only.  No flood depth.
```

One-time CKAN API crawl: 88,327 resources queried (~48 hours). Column headers cached to disk. Available for 48,719 of 26,246 datasets. Zero cost on every subsequent run.

"Building_Damaged_Severe" is unambiguous in a way that "damaged buildings" in a dataset title is not.

Lesson: for distinguishing data content from context, actual column headers outperform any amount of title and tag analysis.

Act 2: Precision Over Scale

Exact RDLS definitions in the prompt - teaching the model the semantic boundary itself

RDLS Component Definitions (from LLM prompt)

Hazard:	The dataset CONTAINS hazard measurements or models. A dataset that merely REFERENCES a hazard event does NOT contain hazard data. Required: measurement values, model outputs, intensity grids.
Exposure:	The dataset CONTAINS spatial inventories of assets at risk. A dataset that mentions affected populations does NOT contain exposure data unless it provides an asset inventory. Required: building locations/counts, population grids.
Vulnerability:	The dataset CONTAINS damage or fragility functions. Required: curves or matrices relating hazard intensity to expected damage rate.
Loss:	The dataset CONTAINS post-event impact data. Casualties, displacement, damage status, economic losses measured after an event occurred.

Finding: with these definitions, Claude Haiku (the smallest, cheapest model) achieved 94% average confidence with effectively zero classification errors. Prompt precision beats model scale for structured classification tasks.

Act 2: Results

7,146 RDLs-relevant records - 3,511 reclassified, 5,448 non-risk separated

Metric	Count	Note	Non-risk domain	Count
RDLs candidates (input)	12,594	From regex pre-screen	General (education, trade, etc.)	2,657
Records sent to LLM	8,506	Ambiguous + validation sample	Health surveillance	941
Reclassified by LLM	3,511	27.9% of input	Reference / admin boundaries	894
Non-risk datasets separated	5,448	43.3% were not risk data at all	Humanitarian operations	263
RDLs-relevant records (final)	7,146	56.7% of input	Climate (non-risk)	39
Schema-valid records	3,312	46.3% of relevant	No HEVL blocks after reconcile	654
Record identifiers renamed	2,425	Reflecting corrected classification		

Cost metric	Value
LLM cost (initial run)	~USD 22
Re-run cost (cached)	USD 0
Cost per record	~\$0.003
Time (with cache)	~22 minutes

LLM overrode the regex result in 61% of validation sample cross-checks (132 of 215 records). Column headers were decisive in the majority of those cases.

Act 2: The Remaining Ceiling

Correct classification - but extraction quality still bounded by regex

Limitation 1: Extraction quality bounded by pattern matching

```
# A 6-return-period probabilistic flood dataset received:
"hazard": {
  "event_sets": [{
    "hazards": [{"type": "flood", "intensity_measure": "wd:m"}],
    "analysis_type": "probabilistic",
    "events": []          # no return periods - regex has no mechanism
  }]
},
"exposure": [{"category": "buildings"}]  # no dimension, no metric
```

Limitation 2: RDLS v1.0 requires fields regex cannot populate

v1.0 Field	What it requires	Regex?
publisher / creator / contact_point	Separate entity objects	Partial
lineage.sources	Per-contributing-dataset provenance	No
attributions	Each contributing org with specific role	No
Multi-event event_sets	One event per stated return period	No
Multiple exposure items	One item per asset category	No
Multiple loss entries	One entry per impact combination	No

Activity 3

The LLM-First Pipeline

One call. Full richness. Zero regex. RDLS v1.0. 26,246 datasets. Zero invalid records.

Act 3: Why Remove Regex Entirely?

Two forces converged: v1.0 schema requirements + economics of LLM pricing

Economics changed

Approach	LLM cost	Time
Act 1: Regex only	USD 0	< 60 sec
Act 2: Regex + LLM	~USD 22	~22 min (cached)
Act 3: LLM all 26,246	~USD 123	~24 hours

USD 123 for 26,246 datasets - comparable to less than one day of contractor time. The economic barrier that motivated the hybrid gating design no longer exists.

v1.0 schema demands LLM-level understanding

`publisher / creator / contact_point`

`lineage.sources` (per contributing dataset)

`Multi-event event_sets` with return periods

`Multiple Exposure_item` per record

`Multiple Losses` entries per record

`impact_modelling`: observed / simulated / inferred

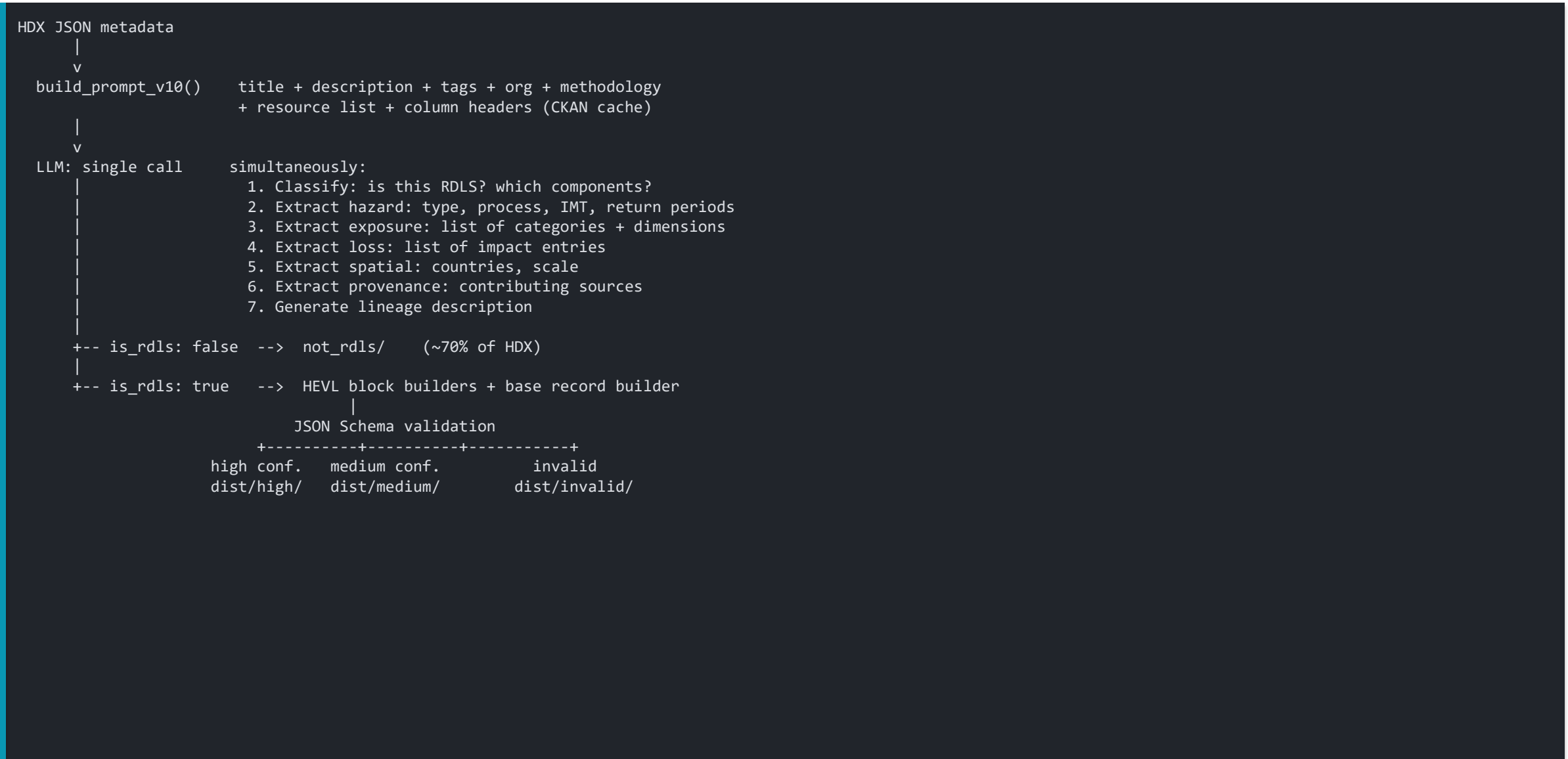
`loss_approach`: empirical / analytical

None of these fields can be populated by regex. They require reading methodology descriptions and making semantic inferences.

Result: a single code path, not four phases. Every dataset takes the same route. Simpler architecture. Higher quality across ALL records.

Act 3: The New Architecture

One LLM call does what four phases previously required



Act 3: Multi-Return-Period Probabilistic Events

Before: empty event set. After: one structured event per return period.

Act 2 output (regex extraction)

```
"hazard": {
  "event_sets": [{
    "id": "event_set_1",
    "hazards": [{
      "type": "flood",
      "intensity_measure": "wd:m"
    }],
    "analysis_type": "probabilistic",
    "events": []
  }]
}

# One empty event set.
# Return periods: not captured.
```

Return periods are extracted ONLY when explicitly stated in the metadata. The LLM is instructed never to invent values - the primary failure mode of regex.

Act 3 output (LLM extraction)

```
"hazard": {
  "event_sets": [{
    "hazards": [{"type": "flood",
      "process": "fluvial_flood",
      "intensity_measure": "wd:m"}],
    "analysis_type": "probabilistic",
    "calculation_method": "simulated",
    "events": [
      {"id": "event_10yr", "occurrence":
        {"probabilistic":
          {"return_period": 10, "event_rate": 0.1}}},
      {"id": "event_25yr", ...},
      {"id": "event_50yr", ...},
      {"id": "event_100yr", ...},
      {"id": "event_200yr", ...},
      {"id": "event_500yr", ...}
    ]
  }]
}

# Six events with probabilities.
```

Act 3: Multiple Exposure Categories + Multiple Loss Entries

Each asset category and each impact type becomes a fully specified block

Multi-category exposure

```
"exposure": [  
  {  
    "id": "exposure_1",  
    "category": "buildings",  
    "asset_type": {"id": "buildings",  
      "description": "Building stock by  
        construction type"},  
    "metrics": [{"dimension": "structure",  
      "measurement":  
        {"quantity_kind": "count"}}]  
  },  
  {  
    "id": "exposure_2",  
    "category": "population",  
    "asset_type": {"id": "population",  
      "description": "Gridded population"},  
    "metrics": [{"dimension": "population",  
      "measurement":  
        {"quantity_kind": "count"}}]  
  }  
]
```

Multi-entry loss

```
"loss": {"losses": [  
  {  
    "id": "loss_1",  
    "asset_category": "buildings",  
    "impact_and_losses": {  
      "impact_type": "direct",  
      "impact_modelling": "simulated",  
      "impact_metric": "buildings_damaged_count",  
      "loss_approach": "analytical"  
    }  
  },  
  {  
    "id": "loss_2",  
    "asset_category": "population",  
    "impact_and_losses": {  
      "impact_type": "indirect",  
      "impact_modelling": "simulated",  
      "impact_metric": "displaced_count",  
      "loss_approach": "analytical"  
    }  
  }  
]}
```

Act 3: Seven Schema Bugs Found and Fixed

Iterative testing against the RDLS v1.0 JSON Schema before production

#	Error observed	Root cause	Fix applied
1	exposure.dimension: 'count' is not one of [...]	LLM returned non-enum values	Validate against closed set; fallback to category default
2	spatial: {scale:'global', countries:[...]} invalid	Lookup fallback + LLM country list combined	Strip countries when scale='global'
3	loss.hazard.type: 'multiple' is not one of [...]	LLM used 'multiple' for multi-hazard datasets	Reject in parser; fall back to primary type
4	resources.0: not valid under any of the given schemas	Empty URL failed Resource.anyOf	Skip resources with no settable URL
5	spatial.countries: ['XKX'] fails codelist	Kosovo code outside 249-entry closed list	Filter all country lists at module import
6	impact_modelling: 'inferred' for probabilistic output	Always defaulted to 'inferred'	simulated/observed/inferred per analysis type
7	loss_approach: 'empirical' for model-based losses	Always hardcoded 'empirical'	analytical for model-based, empirical for field data

None of these errors were obvious from reading the schema specification. Every one was only visible by running the validator against real data. Budget for this iterative cycle before any production run.

Act 3: Test Results - Zero Invalid Records

100-dataset test with Claude Haiku 4.5 after all seven fixes

30 / 30

Schema-valid

0

Invalid

0

Failures

Metric	Value	Improvement over Act 2
Multi-return-period event sets	12 of 30 records (40%)	Was: 0%
Multiple exposure categories	18 of 30 records (60%)	Was: 0%
Multiple loss entries	24 of 30 records (80%)	Was: 0%
Provenance fields populated	30 of 30 records	Was: not supported (v0.3)
LLM cost (100 datasets)	USD 0.47	Scaled: ~\$123 for 26,246
Processing time (100 datasets)	6 min 36 sec	Scaled: ~24 hours for 26,246

Act 3: Model Trade-Off - Haiku vs Sonnet

For 26,246 datasets, model choice has a 3x cost and 3.5x time impact

Attribute	Claude Haiku 4.5	Claude Sonnet 4.6
Input cost	\$1.00 / MTok	\$3.00 / MTok
Output cost	\$5.00 / MTok	\$15.00 / MTok
Avg. call latency	~4 seconds	~11 seconds
Est. full run cost	~USD 123	~USD 370
Est. full run time	~24 hours	~80 hours (3.4 days)
Classification accuracy	Very high	Marginally higher
Extraction richness	Rich	Marginally richer
Practical for bulk run?	Yes	No

Finding: at Haiku pricing, the marginal quality gain from Sonnet does not justify 3x the cost and 3.5x the time for a full corpus run. Haiku is the correct choice.

Solution: use Haiku for bulk + Sonnet only for borderline cases.

Act 3: Recommended Production Strategy

Two runs - Haiku for everything, Sonnet for borderline cases only

Run 1: Haiku for all 26,246 datasets

Model: claude-haiku-4-5-20251001

Cost: ~USD 123 | Time: ~24 hours

Output:

dist/high/	confidence ≥ 0.70	--> ready for publication
dist/medium/	confidence 0.40-0.70	--> borderline cases
not_rdlis/	is_rdlis: false	--> ~70% of HDX

Run 2: Sonnet for medium-confidence tier only

Input: dist/medium/ (~1,500-2,500 records, ~5-10% of RDLS total)

Model: claude-sonnet-4-6

Cost: ~USD 35-60 | Time: ~8 hours

Process: remove medium-tier IDs from progress.jsonl
switch config to Sonnet, re-run pipeline
Haiku high-tier records remain untouched

Total: ~USD 160-180 | ~32 hours across 2 runs

Result: near-Sonnet quality on borderline cases at 45% of full-Sonnet cost

The Journey in Numbers

Three generations - three approaches - one goal

Metric	Act 1: Regex	Act 2: Hybrid	Act 3: LLM-First
Pipeline phases	1	4	1
Datasets processed	12,594	12,594	26,246 (full HDX)
LLM coverage	0%	68%	100%
Phantom components	2,313	0	0
Classification accuracy	~82%	~100%	~100%
Return periods captured	Never	Never	Yes (when stated)
Multi-exposure records	No	No	Yes (60%+ of hazard)
Multi-loss entries	No	No	Yes (80%+ of loss)
v1.0 provenance fields	No	No	Yes
Invalid records (test)	N/A	Present	0
Total LLM cost	USD 0	~USD 22	~USD 123
Re-run cost	USD 0	USD 0	USD 0 (cached)
Schema target	v0.3	v0.3	v1.0

Key Lessons

1. Pattern matching hits a semantic ceiling

Regex works for ~82% of records. It fails when metadata describes context, not content. For HDX, 1 in 5 records crosses that boundary. No additional patterns can fix it.

2. Column headers are the decisive signal

Actual data column headers are unambiguous. 'People_Displaced' in a CSV column is clearer than 'displaced populations' in a dataset title.

3. Prompt precision beats model scale

With exact RDLS component definitions, the smallest Haiku model achieved 94% confidence. Domain-specific definitions matter more than choosing a larger model.

4. Caching makes iteration free

Responses cached by prompt hash. After the first run (~\$123), every code fix, schema update, or output format change runs at zero additional LLM cost.

5. Economics drove the architecture

The shift from hybrid to LLM-first was enabled by cost, not capability. At \$1/MTok, sending 26,246 datasets to the LLM costs less than one day of contractor time.

6. Test against the schema, not assumptions

Seven schema bugs were only visible by running the validator against real data. Budget for this iterative cycle before any production run.

Next Steps

Immediate (production pipeline)

1	Run Haiku for all 26,246 datasets	Pipeline is ready. Config set to Haiku. ~24 hours, ~USD 123. Resumes automatically if interrupted.
2	Run Sonnet on medium-confidence tier	After Haiku completes. ~1,500-2,500 records. ~8 hours, ~USD 35-60. Replaces borderline outputs.
3	Quality review of output	Sample from high/medium tiers. Spot-check multi-return-period events and contributing sources.
4	Semantic validation (Layer 2)	Run cross-field validator. Catch open-codelist and cross-field constraint errors.

Downstream

1. Integrate into RDLS public catalogue and generate search indexes
2. Apply same pipeline to GeoNode, World Bank Data Catalog, other CKAN portals - only the source adapter changes
3. Establish a periodic refresh pipeline for new HDX datasets added since initial run

From Keywords to Understanding

Three generations. One goal: machine-readable risk data at scale.

Approach	Time	Cost	Accuracy	Output quality
Regex (Act 1)	60 sec	USD 0	82%	v0.3, minimal blocks
Hybrid (Act 2)	22 min	~USD 22	~100%	v0.3, single blocks
LLM-First (Act 3)	~24 hours	~USD 123	~100%	v1.0, full richness

Resources

Source code: github.com/bennyistanto/to-rdls

Hybrid pipeline doc: [docs/llm_assisted_metadata_classification.md](#)

LLM-first report: [docs/llm_first_metadata_transformation.md](#)

Technical reference: [docs/llm_first_pipeline_v10.md](#)

RDLS standard: docs.riskdatalibrary.org